

of tables,
and do a Longest Prefix match
to resolve candidates for the glob before validating each one against the
actual value and seeing what matches.

An alternative design here would be to somehow convey this information in
a Rails table: the risk of out-of-sync then becomes minimal and potentially
this table can be used to solve the "option 2" design issue.

Replacing this might be useful anyways, as MemDB requires all entities and
groups fits in memory, but was useful when the storage backend did not
support transactions. Switching to a transactional storage backend would let
us replace this with a LRU cache and more entries on disk and thus have
different performance characteristics if we wanted them. However, we might
still want to allow MemDB users to have support for glob based groups.

Complexity: complex

Impact: necessary performance improvement at scale to reduce the number
of Rails->OpenBao queries.

MVC: not strictly necessary if we don't allow globs in environments,
branches.

GA: likely required if customers have complex environments names

Long-term

The following long-term enhancements are required to add additional features
into GitLab Secrets Manager, but aren't required for the initial set of
designs through GA.

Policy unions

An upstream OpenBao commenter already noted the weird ACL
combining behavior and
wants to address it with strict UNIONS over policies. This isn't useful
for us in the short-term (because all pipelines with separate policies
will have paths with READ access only, and so will all have the same
capabilities), but we'll need this for the future if we support CREATE
or dynamic secrets of various sorts.

Complexity: complex; lots of RFC design work and careful consideration.

Impact: solves the subtoken requirement problem.

Reverse lookup of policies

Notably, the current ACL subsystem cannot handle reverse lookups at all:
given a path, what policies apply to them?

We need an enhancement to build a PathManager
over relevant policies to allow efficient lookup of path->allowing policy.

This could be at a path such as `sys/policies/lookup/acl[:prefix]`, taking a `path=...` parameter conforming to existing PathManager expectations (because it is, after all, an OpenBao path which we want to check access to).

Notably, the current implementation is not serializable, so we will want to handle that so that indices can be persisted to disk (built on update) and not be stored in memory (as we'll likely have lots of policies and hopefully relatively few lookups relatively). A LRU cache can be used to store commonly accessed queries in memory and to provide better locality for fetch.

Complexity: moderately complex to be performant

Impact: solves a use case that required the Rails database.

Exists as an operation

OpenBao supports LIST (and, now, LIST-PAGE), which can be used as proxies for an existence check. However, they have the notable restriction of leaking names of secrets which the user may not have access to. We would like to support scoped-list in the future, but in the mean time, we should also consider a direct existence check, to allow users to make specific queries to specific paths to check if they exist or not. While theoretically equivalent to LIST operations given sufficient volume, they'll allow us to efficiently check (on the client) whether or not hierarchical secrets exist and condition appropriately.

Complexity: Requires some Core knowledge but should be doable.

Impact: low: ways to work around it.

Size limits

This is required for direct user access, and probably notated elsewhere in discussions, but is worth calling out: direct user access will require request size quotas.

Complexity: moderate; much plumbing for other types of quotas already exist.

Impact: large: greatly improves security posture.

Profiles

For many reasons, profiles (server-side execution of multiple requests from a given input and formatted into a response) are an attractive solution to multiple client->server requests.

They are described more in this draft blog post.

JWT direct profiles

For user ACLs, we would have to provision a group for every role and user id when the ACL policy is created. This is fine, but requires several operations to OpenBao. More ideally, we'd create a `policiesclaim`

option on the JWT auth method, which allows a JWT issuer to directly specify ACL policies that the OpenBao token should have post-auth. This would allow us to significantly prune the groups in OpenBao, decreasing that just to the set of wildcard environment/branch scopes.

External transactions

Transactional storage allows

OpenBao to have internal transactional guarantees. However, etcd exposes an external (caller-driven) transaction semantic.

We could add a similar API to OpenBao (though, interactive rather than one-shot) to allow multiple OpenBao requests to be performed transactionally and conflict if modifications occur.

I started down the route of implementing this earlier, however, it requires careful changes in Core that likely would conflict with other changes so I abandoned the approach for the time being. This would make call chaining from GitLab Rails safer.

SECTION: engineering/architecture/design-documents/secret_manager/studies/ci_job_secrets.md

Objectives

- To map out how users can use their native GitLab secrets in their CI jobs.
- Given OpenBao is a fork of HashiCorp Vault, we want to confirm its compatibility with our Vault integration in Runner.
- At a high level, gain a better understanding of how to structure OpenBao policies and JWT roles to be compatible with a project's varied permissions per GitLab user role.

Prerequisites

The workflow requires that the templated policies for each combination of capabilities (e.g. read+update, read+update+create) are predefined. For example, consider the following templated policy that allows full access to a project's secrets:

```
shell
bao policy write projectfullaccess - <<EOF
path "kv-v2/data/projects/{{identity.entity.aliases.authjwt02163755.metadata.projectid}}/" {
  capabilities = [ "read", "create", "update", "delete", "list" ]
}
EOF
```

The policies are associated to JWT roles on authorization. The projectfullaccess policy is particularly important for the initial project owner role:

```
shell
bao write auth/jwt/role/projectowner - <<EOF
```

```
{
  "roletype": "jwt",
  "policies": ["projectfullaccess"],
  "tokenexplicitmaxttl": 60,
  "userclaim": "userid",
  "claimmappings": {
    "projectid": "projectid"
  },
  "boundaudiences": "secrets.gitlab.com",
  "boundclaimstype": "glob",
  "boundclaims": {
    "useraccesslevel": "owner"
  }
}
EOF
```

Given OpenBao policies are deny by default, this initial JWT role is necessary to grant project owners full access to read and write secrets.

Initial setup workflow

Details the steps and technical information for when the project's native secrets are set up for the first time.

1. Project owner enables GitLab Secrets manager through the GitLab UI.
1. Project owner defines additional permissions on which GitLab user roles can read, write, or create secrets through the GitLab UI.
 - By default, project owners have full access and other roles are denied.
 - For example, if the owner allows read-only access for developer role then, through the OpenBao API, the Rails backend defines project88developer:

shell

The format of the role name is project<project-id><user-role>

bao write auth/jwt/role/project88developer - <<EOF

```
{
  "roletype": "jwt",
  "policies": ["projectreadonly"],
  "tokenexplicitmaxttl": 60,
  "userclaim": "userid",
  "claimmappings": {
    "projectid": "projectid"
  },
  "boundaudiences": "secrets.gitlab.com",
  "boundclaimstype": "glob",
  "boundclaims": {
    "useraccesslevel": "developer"
  }
}
EOF
```

- Unlike the projectowner generic role, we have to define other non-owner roles tied to the project because projects may have different combinations of permissions per user role.

1. Project owner defines secrets through the GitLab UI.

- User defines details such as name, key, and value. Sample input:

- name: Production Database Password

- key: DBPASS

- value: mydbpass

- The secret is stored in OpenBao under kv-v2/data/projects/88/ci/DBPASS, with the JSON data:

```
json
{
  "data": "mydbpass"
}
```

- The user doesn't need to enter the secret value in JSON format. The Rails backend transforms the input into JSON object with the data key before sending it to OpenBao.

1. Developer uses the secrets keyword in the .gitlab-ci.yml.

- Sample configuration:

```
yaml
job-with-secrets:
  secrets:
    MYSECRETONOPENBAO:
      key: DBPASS  Translates to kv-v2/data/projects/88/DBPASS, field data
```

- There is no need to specify idtokens:VAULTIDTOKEN as aud defaults to <https://secrets.gitlab.com> where OpenBao service is.

- Unlike with HashiCorp Vault, there is no need to define CI/CD variables.

- The VAULTSERVERURL defaults to <https://secrets.gitlab.com> where OpenBao service is.

- The VAULTAUTHROLE defaults to project<projectid><jobuserrole> to match the JWT role in OpenBao.

1. The CI job runs and MYSECRETONOPENBAO is available as an environment variable.

- OpenBao verifies the integrity of the ID token and validates the boundclaims if it matches the custom claims, specially the useraccesslevel which contains the GitLab user role of the user.

- Similar to HashiCorp Vault secrets, this is a file variable.

Technical implementation findings

High-level technical implementation details pertaining to OpenBao and Rails to support the workflow.

1. The OpenBao service needs to be properly configured to make it compatible with the workflow.

- Configure JWT authentication to make it work with ID tokens authentication.

- The documentation shows instructions using the vault CLI, but it should work similarly for bao.

- The OpenBao API is reachable through <https://secrets.gitlab.com>.
 - To reference the projectid in the templated policy, it was needed to get the value of the JWT auth mount accessor (authjwt02163755 from the result of `bao auth list`). This has to be automated during deployments so that the templated policies remain up-to-date with the correct accessor. The mount accessor value is persisted in storage and keeps its value even when the OpenBao server is restarted and sealed.
1. The Rails backend needs the accompanying implementations to support the workflow.
 - ActiveRecord model for the secrets. Listing secrets and viewing details in the UI shouldn't make a request to OpenBao.
 - ActiveRecord model for the permissions. Listing permissions in the UI shouldn't make a request to OpenBao.
 - Update ID tokens related implementation to support the use of ID tokens without the need to define idtokens in the CI configuration.
 - Proper mapping of defaults for `VAULTSERVERURL` and `VAULTAUTHROLE`.

How to test locally

The policies and roles structure presented here was first tested locally on a GDK setup and OpenBao server running on dev mode.

Here's a step-by-step guide on how to test this locally:

1. Make sure GDK is properly set up with runner.
 - Tested on a GDK with Docker executor and pointed `gdk.test` to `172.16.123.1` but this should also work with a shell executor.
 - Confirm that you can successfully run a CI pipeline on a test project.
1. Create the test project for fetching the secrets from OpenBao later.
 - Track its project ID. In this example, the project ID was 53.
1. Start-up the OpenBao in dev mode.

```
shell
bao server -dev -dev-root-token-id="dev-only-token"
```

- This makes OpenBao reachable at `https://127.0.0.1:8200`.
- You might need to run `export BAOADDR='https://127.0.0.1:8200'` for the `bao` CLI commands below to work.

1. Enable kv-v2 secrets engine.

```
shell
bao secrets enable kv-v2 By default mounts to kv-v2/data
```

1. Enable jwt authentication.

```
shell
bao auth enable jwt
```

1. Configure OpenBao JWT authentication.

```

shell
bao write auth/jwt/config \
  oidcdiscoveryurl="https://gdk.test:3000" \
  boundissuer="https://gdk.test:3000"

```

1. To test the policy and role generated for a project owner with the GitLab user role owner, create the templated policy and the JWT role for the specific owner role. The JWT role was based on the GitLab Vault sample server role.

- Take note of the value of the JWT auth mount accessor when you run `bao auth list`:

```

shell
Path      Type      Accessor      Description      Version
----      -
jwt/      jwt      authjwt02163755  n/a              n/a
token/    token    authtoken90d6d0c1 token based credentials  n/a

```

- define the templated policy and reference the projectid through the metadata of the mounted JWT auth plugin:

```

shell
bao policy write projectfullaccess - <<EOF

owners have full read-write access to their project's secrets
copy over the authjwt02163755 mount accessor value
path "kv-v2/data/projects/{{identity.entity.aliases.authjwt02163755.metadata.projectid}}/"
{
  capabilities = [ "read", "create", "update", "delete", "list" ]
}
EOF

```

- define the JWT role and associate the projectfullaccess policy:

```

shell
bao write auth/jwt/role/projectowner - <<EOF
{
  "roletype": "jwt",
  "policies": ["projectfullaccess"],
  "tokenexplicitmaxttl": 60,
  "userclaim": "userid",
  "claimmappings": {
    "projectid": "projectid"
  },
  "boundaudiences": "secrets.gitlab.com",
  "boundclaimstype": "glob",
  "boundclaims": {
    "useraccesslevel": "owner"
  }
}

```

```
}  
EOF
```

1. Create a sample secret that we want to fetch in the CI job.

```
shell  
bao kv put -mount=kv-v2 projects/53/foo val=my-long-passcode
```

1. On the test project, configure the .gitlab-ci.yml to fetch secrets from OpenBao using the existing Vault integration.

```
yaml  
testopenbao:  
  variables:  
    VAULTSERVERURL: https://127.0.0.1:8200  
    VAULTAUTHROLE: projectowner  
  idtokens:  
    VAULTIDTOKEN:  
      aud: secrets.gitlab.com  
  secrets:  
    SECRET:  
      vault: projects/53/foo/val  translates to secret kv-v2/data/projects/53/foo, field  
val  
  token: $VAULTIDTOKEN  
  script:  
    - echo "testing..."  
    - cat $SECRET  
    - echo "done."
```

- VAULTAUTHROLE matches the JWT role we created earlier.
 - aud matches the role's boundaudiences.
 - The ID token generated in this job is matched by OpenBao using the boundclaims, specifically the useraccesslevel which is included in the custom claims of the ID token.
1. Run a pipeline and confirm that in the job trace there's a masked output of the secret that it fetched from OpenBao.

```
---  
## SECTION: engineering/architecture/design-  
documents/security_analyzer_configuration_profiles/_index.md
```

```
{{< engineering/design-document-header >}}
```

Summary

We need a scalable way of storing and injecting analyzer configuration between security analyzers

and the GitLab monolith. By introducing persisted configuration profiles we can provide an explicit and injectable list of configuration options to be used when executing a security scan.

Motivation

Analyzer configuration was first designed for CI-based scans, but increasingly, GitLab security analyzers are moving outside of pipelines. For CI-based analyzers, we have traditionally relied on exposing CI variables in the corresponding components and templates which are passed to each analyzer for configuring the execution of a scan.

There are several challenges with this approach:

- Difficulty in configuring scans performed outside of CI contexts (e.g. Secret Push Protection and Continuous Vulnerability Scanning)
- Inconsistent configuration profiles between different scan contexts
- Inability to store user-defined configurations uniformly
- Inability to scale shared configuration to organization-wide levels

There's also a growing need to standardize configuration and store user-defined configurations in a consistent and scalable manner.

Goals

Provide GitLab users with a way to:

- Securely store default analyzer configurations within GitLab
- Securely store user-defined analyzer configurations within GitLab
- Manage multiple stored configuration profiles
- Inject configuration profiles into all possible scan contexts
- Validate all possible configuration values for each supported scanner
- Manage configuration profiles at a group and project level
- Get visibility into what has been configured, and where that configuration has been applied

Use cases

Sequence for CI-based scan

mermaid

sequenceDiagram

autonumber

Analyzer->>+Rails: Provide list of configuration options via versioned configuration schema

Rails-->>+Analyzer: Use user-defined overrides + defaults

Analyzer->>Analyzer: Execute scan with configuration profile

Analyzer->>-Analyzer: Generate report with configuration used

Analyzer->>+Rails: Store security report

Rails-->>-Rails: Store scan configuration

Non-Goals

Configuration Profiles are focused entirely on the way in which an analyzer executes its scan. The context by which a scan is ran (i.e. within CI, upon registry push, upon git push, etc..) is out of scope from the initial design.

Proposal

1. Define JSON schema encompassing all supported analyzer configuration options
1. Develop API for persisting and managing user preferences as configuration profiles (validated against schema)
1. Develop UI for persisting and managing user preferences as configuration profiles (validated against schema)
1. Inject chosen configuration profile into scan contexts (starting with CI)
1. Update analyzers to prefer injected configuration settings, prioritizing over ENV and defaults
1. Update analyzers to provide configuration used to execute each scan
1. Generate audit events on configuration profile changes
1. Create both a new default and new custom role for scan configuration profile modification

Configuration JSON schema

Example schema:

```
json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "",
  "title": "Security Analyzer Configuration Options",
  "description": "Mapping of all possible configuration options for Gitlab security analyzers",
  "type": "object",
  "properties": {
    "additionalcacertbundle": {
      "description": "Bundle of CA certs that you want to trust.",
      "type": "string"
    },
    "loglevel": {
      "description": "Set the minimum logging level. Messages of this logging level or higher are output. From highest to lowest severity, the logging levels are: fatal, error, warn, info, debug",
      "type": "string",
      "enum": ["fatal", "error", "warn", "info", "debug"],
      "default": "info"
    },
    "secretdectionhistoricscan": {
      "description": "Flag to enable a historic Gitleaks scan",
      "type": "boolean",
      "default": false
    }
  }
}
```

```
    },  
  }  
}
```

Configuration DB Schema

```
mermaid  
classDiagram  
    class namespaces {  
        id: bigint  
        name: text  
        path: text  
        ...  
    }  
    class projects {  
        id: bigint,  
        name: text  
        path: text  
        ...  
    }  
  
    class securityscanconfigurationprofiles {  
        id: bigint,  
        ...  
    }  
  
    class securityscans {  
        id: bigint  
        createdat: timestamp  
        updatedat: timestamp  
        projectid: bigint  
        scantype: text  
        status: smallint  
        pipelineid: bigint (nullable)  
        buildid: bigint (nullable)  
    }  
  
    securityscanconfigurationprofiles --> securityscans : hasmany  
    securityscanconfigurationprofiles <-- projects : hasmany  
    projects <-- namespaces : hasmany  
    namespaces --> securityscanconfigurationprofiles : hasmany  
    projects --> securityscans : hasmany  
  
---  
## SECTION: engineering/architecture/design-  
documents/security_policies_database_read_model/_index.md  
  
{{< engineering/design-document-header >}}
```

This document is a work in progress and represents the proposal for new Security policies architecture to allow for optimized policy updates and propagation.

Summary

The security policies are stored as YAML files in the security policy project. While this approach has a lot of advantages (like version control for policies using Git, auditable etc), it faces some performance drawbacks.

Since reading from the Git repository requires calls to Gitaly, it might block us in building flexible functionalities.

Motivation

The current architecture of synchronising policies from YAML to approval rules in DB is not very performant as it involves the process of deleting and recreating the approval rules even though the event that triggers the re-sync does not need them to re-process completely (eg: user added to a project would only need to update approvers of approver rules). The current architecture faces these limitations:

- Difficult to build new functionalities
 - Due to the limitation of reading the YAML from Gitaly everytime something gets updated, we keep a limit in the number of policies configured. But the limit is not sufficient for a lot of use-cases.
- High resource consumption
 - Security::ProcessScanResultPolicyWorker is a long-running worker, it makes call to Gitaly, deletes and create approval rules, updates all open MRs for the project. If the project has a huge number of MRs, it could take few minutes to complete, as we currently don't have the possibility to selectively sync a policy.
- Less fault tolerant
 - Since all the operations performed by the worker has to be atomic, if one step fails, it could make the final state of the system to be inconsistent
- Redundant data stored in approval rules table and scanresultpolicies
 - Since we store fields from YAML into approval rules table for all MRs, they are redundant and consume a lot of extra disk space
 - Since we store projectid in scanresultpolicies, we have a huge number of records for a low number of policies that are actually in security policy repository
- Difficulties of fetching the security policies list due to access management
 - Users currently need access to security policy project in order to see the policies applicable to a project where they already have access. This would be solved if we read the policies from the database.

Events that trigger policy synchronisation

The security policies needs to be refreshed whenever these events happen:

- Policy is created/updated/deleted for a project/group
- Project created within a group with policies configured
- Project is transferred to a group with policies configured
- Protected branch is added/removed from a project/group
- Default branch of a project with policies is changed

- Compliance framework label is added/removed for a project
- Cron job that syncs pending policyconfiguration

Goals

1. Reduce the calls to Gitaly and depend on reading from the database. The YAML data will be mirrored on a database table and a read-only ActiveRecord model allows us to build features without performance concerns.
1. Remove duplicated columns related to merge request approval policies in approvalprojectrules and approvalmergerequestrules
1. Change the current process in UpdateOrchestrationPolicyConfiguration not to delete and recreate all related records, but rather update/recreate only the affected records
1. Reduce the average propagation duration of merge request approval policy changes
1. Reduce the number of queries to the database performed by workers from Security Policies

Proposal

Current architecture of Security policies

Security policy project that are linked to a group or a project has a corresponding entry in securityorchestrationpolicyconfigurations table. In order to read/query policies, the YAML has to be read from policy project which involves a RPC call to Gitaly. This does not perform well when there are huge number of projects/groups configured with security policies.

Merge request approval policy that are stored in the policy project as YAML file gets synchronised to approval rules through approvalprojectrules table. Currently we are storing these fields related to merge request approval policy in the table:

- scanners
- vulnerabilitiesallowed
- severitylevels
- reporttype
- vulnerabilitystates

The same fields are also duplicated in approvalmergerequestrules to relate them to each individual merge request. They are redundant as any value for these fields changed in approvalprojectrules results in approvalmergerequestrules getting updated too.

Security::ProcessScanResultPolicyWorker is responsible for reading the policy from policy project and transforming them to approval rules. The worker performs these operations in sequence:

- Delete approvalprojectrules and approvalmergerequestrules
- Read YAML from policy project (call to Gitaly)
- Convert the YAML to create rows in approvalprojectrules
- Update each of approvalmergerequestrules for open merge requests

We delete and recreate approval rules because we cannot get the exact diff of the policy from YAML very efficiently as it involves call to Gitaly and also we do not have unique identifier for each policy in the YAML.

To avoid this, we have created the scanresultpolicies table (Security::ScanResultPolicyRead model) which acts as a read model for merge request approval policies to avoid reading from policy project. But currently, we don't store all the required fields in the table, we only store roleapprovers , licensestate and matchoninclusion (previously matchoninclusionlicense).

mermaid

erDiagram

```
securityorchestrationpolicyconfigurations ||--o{ scanresultpolicies : " "  
securityorchestrationpolicyconfigurations ||--o{ approvalprojectrules : " "  
securityorchestrationpolicyconfigurations ||--o{ approvalmergerequestrules : " "  
scanresultpolicies ||--|| approvalprojectrules : " "  
scanresultpolicies ||--|| approvalmergerequestrules : " "
```

```
securityorchestrationpolicyconfigurations {  
    int projectid  
    int namespaceid  
    string securitypolicymanagementprojectid  
}  
scanresultpolicies {  
    int securityorchestrationpolicyconfigurationid  
    int orchestrationpolicyidx  
    text licensestates  
    boolean matchoninclusion  
    int roleapprovers  
    int agevalue  
    smallint ageoperator  
    smallint ageinterval  
    jsonb vulnerabilityattributes  
    int projectid  
    jsonb projectapprovalsettings  
    smallint commits  
}  
approvalprojectrules {  
    int scanresultpolicyid  
    int securityorchestrationpolicyconfigurationid  
    int orchestrationpolicyidx  
    text scanners  
    smallint vulnerabilitiesallowed  
    text severitylevels  
    smallint reporttype  
    text vulnerabilitystates  
}  
approvalmergerequestrules {  
    int mergerequestid  
    int scanresultpolicyid  
    int securityorchestrationpolicyconfigurationid  
    int orchestrationpolicyidx  
    text scanners  
    smallint vulnerabilitiesallowed  
    text severitylevels
```

```

    smallint reporttype
    text vulnerabilitystates
}

```

Proposed Architecture

To solve the challenges and limitations mentioned above, we need to persist all fields from the policy YAML to DB (in securitypolicies table) which can be used instead of reading from YAML in Git repository.

The DB schema should closely mimic the policy YAML and should look like this:

```

mermaid
erDiagram
    securityorchestrationpolicyconfigurations ||--o{ securitypolicies : " "
    securitypolicies ||--o{ scanexecutionpolicyrules : " "
    securitypolicies ||--o{ approvalpolicyrules : " "
    securitypolicies }o--o{ projects : "via securitypolicyprojectlinks"
    approvalpolicyrules }o--o{ projects : "via approvalpolicyruleprojectlinks"
    approvalpolicyrules ||--|| approvalgrouprules : " "
    approvalpolicyrules ||--|| approvalprojectrules : " "
    approvalpolicyrules ||--|| approvalmergerequestrules : " "
    approvalpolicyrules ||--|| softwarelicensepolicies : " "
    approvalpolicyrules ||--|| scanresultpolicyviolations : " "

    securityorchestrationpolicyconfigurations {
        int projectid
        int namespaceid
        int securitypolicymanagementprojectid
    }
    securitypolicies {
        int securityorchestrationpolicyconfigurationid
        int policyindex
        text checksum
        text name
        text type
        text description
        boolean enabled
        jsonb policyscope
        jsonb metadata
        jsonb content
    }
    projects {
        int id
        text name
    }
    scanexecutionpolicyrules {
        int securitypolicyid
        int ruleindex
    }

```

```

    text checksum
    jsonb content
  }
  approvalpolicyrules {
    int securitypolicyid
    int ruleindex
    int type
    text checksum
    jsonb content
  }

  approvalgrouprules {
    int namespaceid
    int approvalpolicyruleid
  }
  approvalprojectrules {
    int projectid
    int approvalpolicyruleid
  }
  approvalmergerequestrules {
    int mergerequestid
    int approvalpolicyruleid
  }
  softwarelicensepolicies {
    int projectid
    int approvalpolicyruleid
  }
  scanresultpolicyviolations {
    int projectid
    int mergerequestid
    int approvalpolicyruleid
  }
}

```

In order to achieve this, we want to introduce a new worker(`Security::PersistSecurityPoliciesWorker`) that reads the YAML from Git repository and convert them to entries in `securitypolicies` table, together with the underlying `scanexecutionpolicyrules` and `approvalpolicyrules` tables. This worker should be called only when a policy is created/updated/deleted. In all the other places where we currently read the YAML from Git repository, we should read from `securitypolicies` which serves as SSoT for the latest version of the policies.

This allows us to:

- Know what updated in a policy by comparing the updated value in YAML from values in the table
- Improve performance by adding DB index
- Reduce redundant data stored in `approvalmergerequestrules` and `approvalprojectrules` tables, thereby reducing the DB size
- Dramatically reduce the number of `scanresultpolicies` rows by removing the `projectid` column and creating the link between the projects via a join table

- Reduce redundant data stored in scanresultpolicies, as we the rule data would be decomposed and we wouldn't have duplicated projectapprovalsettings, thereby reducing the DB size

There is an ongoing effort in Allow group-level MR approval rules for 'All Protected Branches' that would allow us to define approvalgrouprules for group policies. This means we will not need to copy approvalprojectrules for every project in a group, reducing the number of workers needed to propagate policy changes.

Design and Implementation Details

Step 1: Add new tables

As a first step, we need to introduce securitypolicies, approvalpolicyrules, scanexecutionpolicyrules, securitypolicyprojectlinks and approvalpolicyruleprojectlinks tables. These tables and columns map to the fields in YAML.

Step 2: Introduce new worker to sync policies to the DB tables

The new worker would be responsible for reading and converting the YAML to rows in securitypolicies table.

Using checksum, we can determine whether a policy has changed and requires a rebuild, or whether it was only re-ordered. We also need to compute the policy diff with created, updated, deleted and re-arranged policies in an efficient data structure for efficient processing of policy changes.

We map a policy to a project if the policy is enabled and the policyscope is applicable to a project by creating a row in securitypolicyprojectlinks table. Using this table we can query the policies that a given project is applicable to.

We also map the approvalpolicyrules (that are defined in the rules field of the policy) to project through approvalpolicyruleprojectlinks table.

Step 3: Migrate all existing policies to securitypolicies table

We need to introduce DB migration that reads all existing policies and populate securitypolicies, approvalpolicyrules, scanexecutionpolicyrules, securitypolicyprojectlinks and approvalpolicyruleprojectlinks tables

Step 4: Introduce a new worker to sync policy from securitypolicies

Once all the policies are migrated to the database, we want to introduce a new worker to sync policy from the table. Security::ProcessScanResultPolicyWorker syncs all policies for a project, but in this step the new worker will take securitypolicyid and policy changes or event type as arguments to selectively sync the approval rules associated to the securitypolicy.

Step 5: Migrate existing approval rules tables to set approvalpolicyruleid

After all the policies are migrated, approvalpolicyrules table would store the information that are needed for approval rules related to policy. But for older approval rules records, approvalpolicyruleid would be nil. In this step, we want to introduce a background migration to backfill approvalpolicyruleid for approvalprojectrules, approvalmergerequestrules and approvalgrouprules tables.

Step 6: Use approvalpolicyrules table instead of scanresultpolicies

Once all policies are migrated, we want to use approvalpolicyrules to query the fields that are related to policy instead of reading from scanresultpolicies table.

Step 7: Delete columns from approval rules table

This step would delete the columns that are migrated to securitypolicies from approvalprojectrules and approvalmergerequestrules.

Step 8: Remove scanresultpolicies table

At this point, we can remove the old table scanresultpolicies because the approval rules would be linked via approvalpolicyrules table.

Links

- Use database read model for scan result policies
- Spike: Prepare architecture blueprint for database read model for scan result policies

SECTION: engineering/architecture/design-documents/self_hosted_finetuning/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- This renders the design document header on the detail page, so don't remove it-->

{{< engineering/design-document-header >}}

Summary

This blueprint proposes a solution for fine-tuning Duo Self-hosted models using Parameter-Efficient Fine-Tuning (PEFT).

Goal

Enable lightweight and efficient model fine-tuning for GitLab Duo Self-hosted customers.

Non-goals

Any other forms of customizing the model:

- Full model fine-tuning
- RAG
- RLHF

Motivation

With the development of Duo Self-hosted, the need for model customization has arisen. As the

first example, customers were dissatisfied with the performance of the supported models on Code Suggestions for some niche coding languages. Another example was the desire for more personalized code suggestions, i.e. feature responses that are more accurate to their requirement and follow the coding patterns of their codebase. One of the possible approaches to customize the model is to fine-tune it for a specific task or user's codebase.

Challenges of Model's Fine-tuning

While fine-tuning the entire model is a solution for model customization, it comes with several big challenges:

- 1) Training a model usually requires roughly 3-4x more vRAM than simply loading the model.
- 2) Storing n custom fine-tuned models would have high disk usage.
- 3) Serving n different fine-tuned models simultaneously might be slow and resource expensive.
- 4) Training a model requires extensive and specific knowledge, reinforcing the need to enable customers with a user-friendly UI.

This, together with the overall expectation that an average customer would be limited in its available hardware resources, motivates us to look into other more efficient approaches.

PEFT and light-weight adapters

The solution selected to achieve lightweight and quick fine-tuning is using PEFT techniques; specifically the use of adapters.

What is an adapter?

Adapter-based methods add extra trainable parameters to the existing (base) model's layers. The base model weights stay frozen, while the new additional weights are trained on a new dataset. In the case of Code Suggestions, the dataset could be the customer's codebase or any other suitable data. One of the most widely used adapter-based methods is Low-Rank Adaptation (LoRA). In a nutshell, LoRA uses small rank matrices that are combined with the original model's weights. These small new weights are stored separately and are a magnitude smaller in size than the weights of a base model. During the inference, the new weights are combined with the base model weights, allowing us to simultaneously host one base model and multiple different task-specific LoRAs.

Additional Resources:

- LoRA: Low-Rank Adaptation of Large Language Models
- A Survey on LoRA of Large Language Models

Advantages and Limitations

As with any method, the LoRA method comes with its advantages and limitations. Generally, LoRA slightly loses against a fully fine-tuned model in the overall performance; if time and hardware are the restrictions, however, then a small performance degradation may be acceptable.

Advantages:

- 1) Training an adapter requires less memory and is much more time-efficient than full-model fine-tuning
- 2) The adapter's size would be in 100s of megabytes rather than 100s of gigabytes, simplifying storage and enabling easier use of multiple adapters, though the base model is still needed.
- 3) Hosting and switching between different adapters is usually a low-latency procedure

Limitations:

- 1) LoRAs tend to "forget things" a bit more than the fully fine-tuned model. Thus, we can expect LoRA to perform better than the base model on the task for which it was trained, but it could perform worse than a base model on the tasks for which it was not trained.
- 2) LoRAs are model-specific, meaning LoRAs can only properly work on the model it was trained on.
- 3) LoRA is likely to lose in the overall performance and generalization to a fully tuned model.

Design and Implementation details

Training

```
mermaid
graph LR
    subgraph s2["UI"]
        project1["Project 1"]
        project2["Project 2"]
        project3["Project 3"]
    end
    dataprep["Data Preparation"]
    data1["Data 1"]
    data2["Data 2"]
    data3["Data 3"]
    combineddata23["Data 2 + 3"]
    training["Adapter Training"]
    adapter1["Adapter 1"]
    adapter2["Adapter 2 + 3"]
    modeleval["Model Evaluation"]
    inference["Inference"]
    project1 --> dataprep
    project2 --> dataprep
    project3 --> dataprep
    dataprep --> data1 & data2 & data3
    data2 --> combineddata23
    data3 --> combineddata23
    combineddata23 --> training
    data1 --> training
    training --> adapter2 & adapter1
    adapter1 --> modeleval
    adapter2 --> modeleval
    modeleval --> inference
```

Launching Fine-tuning in the UI

As a first step for adapter training, the user would select a project or a collection of projects to use for fine-tuning in the UI. Once selected, the user will configure and launch the fine-tuning pipeline.

The configuration page will allow the user to:

- Confirm the projects and select the file types for which the adapter will be trained
- Configure where the data will be stored and provide necessary keys (if stored remotely)
- Configure where the adapters weights will be stored
- Change the hyperparameters via sliders or fields (defaults will be provided by GitLab)

The fine-tuning pipeline would then deploy an instance of the fine-tuning service on their configured infrastructure and trigger the following steps:

- Data Preparation
- Adapter Training
- Evaluation

The fine-tuning service would be provided as a Docker image.

The container will be published in the GitLab.com Container Registry and DockerHub regularly.

Data Preparation

Once the fine-tuning pipeline is triggered and service has been deployed, it would start with preparing the data.

The Data Preparation step of the pipeline would process the data provided by the selected repositories, constructing both a training and a validation dataset out of it and storing them on the hard disk based on the provided configuration. The data could be stored either locally or on third-party storage solution (i.e. AWS S3), as long as it is supported and accessible by the fine-tuning service. To speed up training, data will be encoded and stored in vector format.

At the initial version of the service we will support local storage and AWS S3.

Adapter Training

Once the data is ready, the next step in the pipeline is to train an adapter for the given data and provided configuration. The fine-tuning service would fetch the data and use that data as the basis for training. The user would be provided with the feedback on the training, such as evaluation and training performance and the remaining time.

The fine-tuned service will be written using well-known HuggingFace libraries and PyTorch and the weights will be stored in the widely accepted and used HuggingFace format.

Fine-tuned Model Evaluation

Once the adapter is trained, the next step in the pipeline would evaluate it in terms of the

overall performance and responses.

The evaluation step would use a small random sample from validation dataset to test the fine-tuned model and present the results to the customer.

Under the hood, the evaluation step will deploy a newly fine-tuned model using HuggingFace libraries and run the model on the prompts from the validation datasets. The model's outputs will then be evaluated against the ground truth responses using Cosine Embedding Distance.

Inference

```
mermaid
graph LR
    subgraph vllm["vLLM"]
        basemodel["Base Model"]
        adapter1["Adapter 1"]
        adapter2["Adapter 2"]
        adapter3["Adapter 3"]
    end

    subgraph ui["UI"]
        project1["Project 1"]
        project2["Project 2"]
        projects345["Project 3-5"]
        project6["Project 6"]
    end

    basemodel --> adapter1
    basemodel --> adapter2
    basemodel --> adapter3
    adapter1 --> project1
    adapter1 --> project2
    adapter1 --> projects345
    adapter3 --> project6
```

Fine-tuned Model Deployment

Once the task-specific adapters are trained, customers would need to host their base model and trained LoRAs using vLLM.

vLLM supports hosting and inference of base model and LoRAs out-of-the-box. To deploy the base model with its adapters, the user would need to provide the paths to the stored adapter's weights.

To trigger the inference with a specific adapter, the customer could specify the adapter's name in the API request to vLLM. The vLLM will do all the heavy lifting behind the scene, by loading and merging the adapter weights into the base model.

Fine-tuned Model and UI during the Inference

```
mermaid
graph LR
    subgraph ui["UI"]
        project1["Python Project"]
        projectconf["Project Configuration"]
        uiapi["API"]
    end
    aig["AI-Gateway"]
    vllm["vLLM"]
    apirequest1["{ model: lora-python, prompt: def helloworld }"]
    apirequest2["{ model: lora-python, prompt: def helloworld, maxtokens: 64, temperature: 0 }"]
    projectconf --> project1
    apirequest1 --> aig
    aig --> apirequest2
    apirequest2 --> vllm
    project1 --> uiapi
    uiapi --> apirequest1
    vllm -- def helloworld(): print Hello World! --> aig
    aig -- print Hello World! --> uiapi
```

Each adapter could be used with either a single project or a collection of projects. It is also true that one project could use several adapters at the same time. The UI will provide a way for the user to select which adapter to use for which project. The UI will also inform the user which adapters were trained for which project. By default, each project would be assigned a base model.

If the project is new (i.e. there isn't much code to fine-tune on), the user could still pick an adapter from already existing ones. By default such project will also be assigned a base model.

During the feature request the configured model will be used for the inference by the backend.

Technical Details and Early results

Technical Details

Storage per Adapter: 200Mb - 1GB. The size of 1 LoRA adapter could vary based on the configuration.

Training Time per Adapter: 30 minutes to 1 hour (Tested on 4xA100 80GB GCP server). Time varies based on the size of the training dataset. In the chart below, we present the results for the embedding cosine similarity vs training time. Here "0" minutes represents the base model without any fine-tuning. Red line is the results for codesuggestionsaigsignatures and blue line for code-suggestions-input-testcases-v1 datasets, respectively.

mermaid

xychart-beta

title "Embedding cosine similarity vs Training time"

x-axis "Time in minutes" [0, 6, 12, 18, 24, 30, 36, 48, 60, 72, 84]

y-axis "Embedding cosine similarity (higher better)" 0.7 --> 0.95

line [0.75, 0.80, 0.84, 0.83, 0.85, 0.82, 0.85, 0.83, 0.83, 0.85, 0.84]

line [0.85, 0.89, 0.91, 0.90, 0.90, 0.90, 0.91, 0.90, 0.90, 0.90, 0.90]

mermaid

xychart-beta

title "Bad Responses vs Training time"

x-axis "Time in minutes" [0, 6, 12, 18, 24, 30, 36, 48, 60, 72, 84]

y-axis "% of Bad Responses (lower better)" 0.0 --> 0.35

line [0.0, 0.3, 0.07, 0.06, 0.06, 0.03, 0.0, 0.0, 0.0, 0.0, 0.0]

line [0.0, 0.27, 0.05, 0.04, 0.04, 0.03, 0.0, 0.0, 0.0, 0.0, 0.0]

While we see the relative performance increase after only 6 minutes, the model is still unstable and requires further training time to reduce the number of empty responses.

As a conclusion, it seems to be optimal to train the model for 30-40 minutes.

Hardware Specs for Training an Adapter: This depends on the selected base model. For Codestral-22B, the minimum spec is 4xA10, while recommended spec is 4xA100 80GB GPUs. In total the current set up uses 242GB of vRAM.

Early Experimentation (PoC) results

A PoC has been developed for Code Suggestions (code generation and completion) feature. The adapter was trained for ai-gateway for Codestral-22B. The fine-tuned model has shown positive results on both manual and automated evaluations.

For manual evaluations, the model has been deployed and tested with GitLab Duo Self-Hosted in the WebIDE. The fine-tuned model proposed code suggestions that are more aligned with the overall code structure than the base Codestral-22B model:

Results for manual evaluations in WebIDE and code completion results

In addition to the manual evaluations, the fine-tuned model was evaluated on several datasets where it also showed positive results (i.e. code suggestions were more aligned with existing code).

Results for Code Completion using ELI5

In the table below, we present the results of evaluating the fine-tuned model and base model on three different datasets for a code completion task.

In the columns, the two numbers are: embedding similarity, exact match. Higher number is better (meaning results were more similar to the expected output).

Model	code-suggestions-input-testcases-v1	codesuggestionsaigrandomfim	codesuggestionsaigsignatures
-----	-----	-----	-----
Codestral-22B	0.89, 0.03	0.84, 0.0	0.80, 0.0
LoRA+Codestral-22B	0.91, 0.17	0.87, 0.0	0.85, 0.05

Other models to be assessed: Mistral Small 3.

Alternatives

Adapters, and in particular, LoRAs are not a panacea; it is one of the methods that we could utilize if it is found suitable for our use cases. Other potential approaches are:

- 1) Full fine-tuning of a smaller model
- 2) Human-feedback (RLHF)

Smaller experts.

Fine-tuning a smaller model might be a suitable approach, as the time and hardware resources to train such a model are better than a model with a high number of parameters. However, the scalability of such an approach is questionable, as well as the overall performance of a smaller model.

Pros:

- Small LLMs would be relatively quick to fine-tune.
- Fully fine-tuned model generalizes better than LoRA.

Cons:

- Less scalable than LoRA. Hosting several models at the same time is resource expensive.
- Generally less capable than big LLMs.
- Potentially more resource expensive than LoRA.

Reinforcement Learning from Human Feedback (RLHF).

Human feedback could be used together with reinforcement learning (RLHF) to further fine-tune the model to the user's needs. For example, whether a suggestion was accepted or rejected can serve as valuable domain-specific feedback data to then further customize the model's fine-tuning process.

Pros:

- Utilizes human feedback, fine-tuning the model to the real user needs.
- Continuous learning.

Cons:

- Usually requires a lot of data. The approach would be more suitable for .com rather than

Self-hosted.

Why Adapters?

As discussed before, the main challenge of fine-tuning on the customer's infrastructure is to make the process quick and resource efficient. Adapters allow us to fine-tune the model quickly and relatively cheap. In addition, adapters are scalable and lightweight allowing us to potentially host hundreds of them on the single GPU-capable instance. Adapters are widely used in the industry, where quick and lightweight fine-tuning is required for example Apple Intelligence) and Github Copilot.

SECTION: engineering/architecture/design-documents/service_catalog/_index.md

{{< engineering/design-document-header >}}

Summary

Scalability themes:

- Observability across the Production Fleet is accessible for all
- Paved roads are the default for all team members

This document describes how we want to evolve our metrics- and service-catalog into a single definition in YAML for each service. We will have a single source of truth describing a service. The YAML is inspired by a Kubernetes kind with an associated spec that can easily be validated.

The service catalog entry should be the frontend for everyone that wants to know or adjust something to a service's ownership, deployment, monitoring & alerting and deployment properties.

The static YAML that is the services catalog becomes the frontend with which service owners interact to configure multiple backends. These backends consume the YAML to generate alerts, dashboards and recording rules.

It is the foundation for
&1355.

Glossary

Name	Description
----- -----	-----

Service-Catalog Frontend	The static YAML containing service definitions that people interact with.
Service-Catalog Backend	A single responsibility consumer of the service catalog:

generating dashboards, alerts, recording rules, ... |

Motivation

We currently have 2 definitions for a service: one in the metrics-catalog, and one in the service-catalog.

The metrics-catalog is defined in Jsonnet and contains everything needed for monitoring the service and routing alerts to the relevant groups. This includes labeling information, deployment type, and ownership information.

The service-catalog is currently defined as a single YAML file that lists all services with information about labeling, ownership, links to documentation and monitoring resources.

This means a lot of information is duplicated. We need to define a service in both places, and it is easy to forget one or the other.

Furthermore, because the metrics-catalog is in Jsonnet, this makes it hard for anyone to contribute to. They need to familiarize themselves with the language and our DSL to be able to add or modify SLIs.

By unifying both definitions into a static YAML frontend, we make this more accessible to everyone at GitLab first. GitLab.com, Cells for GitLab.com, GitLab-Dedicated tenants and the many Self-Managed deployments can have different service-catalogs that backends consume to generate monitoring resources.

In the future we can add more purpose-built backends using the service catalog.

We have made it much easier to deploy a new service to production using Runway. But it is still a manual step to add it to the service-catalog & metrics-catalog to actually have the new service observable. With

this, we'll cut that down to a single service entry, and it paves the way to use the service catalog as the Single Source of Truth for runway services in favour of the service-inventory that is now required for Runway (runway122).

This also opens up the path to productizing our monitoring of GitLab. Because we ourselves will have used different service-catalog definitions to monitor GitLab.com and GitLab Dedicated. We could look into exposing a catalog as a default configuration for a monitoring-mixin (scalability2832). Even though this is listed as a Non-Goal below, we will pay attention that when we do this project, don't introduce any changes that make this

more difficult in the future.

Goals

1. Make it easier for everyone to contribute to metrics-catalog. So changing SLOs and SLIs is no longer daunting.

Expected impact: Service owners can take ownership of the observability of their service with minimal involvement from Scalability-Observability.

1. Service onboarding is much easier for people that have deployed new services using Runway. Only a single entry is needed in a static file.

Expected impact: All production runway services have entries in the new service catalog. As a result, they'll always at least have a default dashboard.

1. Make it easier for incident responders to see raw metrics associated with an alert.

Expected impact: Faster incident mitigation and reduced stress for the folks on-call.

1. Decouple the generation and deployment of dashboards, recording- and alerting rules from the service definition.

Expected impact: Changes to dashboards, recording rules and services become easier to review for the maintainers of Runbooks and spun off projects with clearer ownership.

1. Avoid having services that have no monitoring resources at all. Or alerts that have no entry in the service catalog.

Expected impact: Less surprises with unmonitored services or problems discovering service owners.

1. Make our metrics stack easier to iterate on by decoupling the service definitions from rule- and dashboard generation.

Expected impact: Make it easier to do projects like Covered Experience SLIs and Upgrading our grafonnet dependency easier to pick up and parallelize.

Non Goals

1. Removing Jsonnet from the process that currently generates recording rules, alerts and dashboards.

1. Make it possible to reuse our monitoring resources for self-managed installations. Though this paves a way for doing something like exposing a monitoring mixin (2832), it is not an immediate goal of this effort.

Implementation phases

To get to the desired goal of having a YAML file for the service definition, we could iterate in the following phases:

1. Generate the static file per service based on the content that is currently in the existing service-catalog and metrics-catalog.

In this stage, we'll add another step to the generation in the runbooks repository. It will generate the file that we want service owners to interact with in the future.

We'll determine the spec that for the file, and add validation for that spec. The spec will be documented and kept up-to-date in this blueprint.

Result: The people reviewing runbooks MRs could now use the diff in this static file to better understand what SLIs are introduced and what SLO applies to it.

1. Consume the generated service-definitions in new projects to generate rule files, alerts, and dashboards. This is where we start moving things out of the runbooks repository. The things required to generate those service definitions from Jsonnet + YAML are all that remain.

At this moment, it's possible that we need to adjust our pre-defined schema because something no longer makes sense.

Result: When we move things out of the runbooks, we can improve the Jsonnet code. We can extract shared responsibilities out into libraries that can be reused in multiple repositories used for generating dashboards or rule files.

Dashboards and their customization now live in a different repository from the Runbooks repository.

Changes to SLIs and SLOs remain in the runbooks' Jsonnet files in this step.

1. Remove the Jsonnet and old service-catalog.yml from the runbooks repository. We'll have to update any other consumers of the service catalog to use the new service definitions.

Result: Service owners now have a single, static service

definition to maintain.

Each of these phases is one or more projects that can be worked on with Scalability-Observability. Each delivering a small portion and allowing us to reassess priority between each project.

SECTION: engineering/architecture/design-documents/siphon/_index.md

{{< engineering/design-document-header >}}

Summary

This document describes the architecture of the MVP version of Siphon, which delivers serialized CDC (change data capture) data from the PostgreSQL logical replication stream to a queueing system. From the queueing system, consumers can process the data and ingest it into other database systems (e.g. ClickHouse, Snowflake).

Project goals

- Replace existing data synchronization tooling by providing a standard way of getting data out of the PostgreSQL databases.
- Asynchronously streaming data change events from our PostgreSQL databases via pub-sub.
- Possibility of extracting data from existing tables in a consistent manner.
- Intended to be deployed through Kubernetes environment on GitLab.com, Dedicated and Self-Managed through the Cloud-Native GitLab Charts (and Omnibus).
- The tool is mostly transparent to application developers making changes to the Rails application.
- Zero or minimal impact on the availability of the PostgreSQL cluster.
- Must support Self-Managed Postgres version 14 and above clusters (with and without Patroni). Additionally it must support Cloud-Managed Postgres Clusters via Amazon RDS and Google CloudSQL, using only features and extensions available in these managed services.

Project non-goals

- The system will only support PostgreSQL replication. Other types of data changes such as Redis or Git changes will not go through the Siphon application. The queueing system for such changes might be the same.
- The system will not allow the application to emit custom events. All events will originate from the Siphon producer which are logical replication events or row data from the initial data snapshot process. The origin of the data will be always the PostgreSQL database.

Main components

The main components of Siphon are:

- Producer application (single-binary application):
 - Manages a PostgreSQL publication.
 - Reads data from the replication slot.

- Orchestrates an initial data snapshot from existing tables.
- Serializes and enqueues CDC events.
- Pluggable Queueing/PubSub system (TBD, we're evaluating NATS)
 - Durable data store.
 - Provides at-least-once delivery guarantees. (to be verified if we require exactly-once delivery guarantees)
- Capable of storing/buffering large tables (2-3 terabytes).
- Pluggable configuration with the possibility of supporting different queueing systems.
- Consumer application (single-binary application):
 - Connects to a target database system.
 - Reads CDC events from the queue.
 - Reformats the events and pushes the data to the target database system.

Source

Motivation

The PostgreSQL database in gitlab-rails is not optimized for analytical workloads. Over the years, teams (Optimize, Analytics) have faced several challenges when architecting scalable analytical features. While these features work, they require substantial engineering investment (background aggregations, denormalization), significantly affecting team velocity and often leading to optimizations specifically catering for the OLTP database, rather than our ability to provide data analytics.

Additionally, running analytical workloads in our OLTP database has led to numerous production incidents over the years, and adds unnecessarily to production load.

By adopting the ClickHouse analytical database, we now have a scalable solution to support the existing analytical features within GitLab. The main challenge becomes data synchronization. Currently, we have at least 3 distinct methods to synchronize data from PostgreSQL to ClickHouse. These solutions are often tailored to specific problems, making generalization difficult (periodical workers to send data, consistency workers to ensure data consistency). Some of these synchronization strategies are implemented specifically for GitLab.com. Establishing a unified approach to transferring data from PostgreSQL to other database systems will greatly simplify and streamline our data synchronization processes, reducing fragmentation and enhancing overall efficiency.

With a tool like Siphon, we can efficiently make PostgreSQL database tables available in other database systems with minimal lag.

Other possible use cases for Siphon:

- Contributing towards to the data unification effort by consolidating all data synchronization implementations from the PostgreSQL database.
- Replicating the production database to Snowflake for the data team.
- Implementing advanced analytics capabilities for all GitLab features beyond Optimize and Platform Insights.
- Real-time threat and abuse detection by the Trust and Safety team with their Omamori project.
- Building GitLab features that can react asynchronously to database changes.

- Providing customers a way to stream data changes.

Alternative options

We have considered the following tools:

- PeerDB
 - It was recently acquired by ClickHouse and they give less or no focus on other data warehouses.
- Debezium
 - Debezium requires Kafka (this can be a problem for self-managed instances) and brings in a lot of new dependencies.
- Airbyte
 - Significant limitations, including no support for schema changes.

We haven't found a tool that supports gradual rollout by adding tables one at a time to a PostgreSQL publication with initial data snapshotting capabilities. During the PoC phase, we found a procedure that achieves this in an eventually consistent manner, assuming that we provide at-least-once delivery guarantees (allowing for duplicated events).

Infrastructure Diagram

This diagram shows a deployment for GitLab.com, where we have one consumer application ingesting data into ClickHouse. The MVP will also include another consumer application for ingesting data into Snowflake.

Source

Future state: multiple cells

Source

In a multi-cell environment we will have two kinds of consumers:

- Global consumers: can receive data from all cells.
- Local consumers: can only read queues from the cell where they are deployed.

Note: The setup might also include cell-local queueing systems where a special consumer would push data from the cell-local queueing system to a global queueing system (GitLab.com only).

Producer Application Components

Source

- Lock goroutine: Ensures that only one instance of the application is connected to the PostgreSQL DB with the given application id (configuration). It uses advisory locks (related issue).
- Logical replication goroutine: Reads the logical replication stream and pushes data to the main buffer.
- Initial data snapshot goroutine: Captures data from existing tables. This goroutine is mostly idle after initial data snapshot is done.
- Monitor goroutine: Periodically collects statistics.

Initial data snapshot procedure

When creating the publication for the first time, as the first step, the application will need to read the existing data from the database tables. This requires one or more extra connections to PostgreSQL.

A high level process for the initial data snapshot (for one table) can be the following:

1. Ensure that the configured PostgreSQL server for the initial data snapshot is caught up to the LSN that altered the PUBLICATION (table added).
1. Inspect and determine the table size and the number of blocks it uses.
1. Using the number of blocks, we can determine N ranges which can be processed by N threads (can be configured).
1. Read the data in the batches, serialize and send the payload to the queueing system. Using efficient TID range scan.

The first step is important to ensure consistency in case we use a second physical standby server for the initial data snapshot.

During the initial data snapshot all records will be INSERT statements, the order in which we send the batches to the queueing system does not matter. While the initial data snapshot is taken, we must not enqueue any data from the logical replication stream for the given table.

To take a consistent snapshot from the PostgreSQL database we would need to export a snapshot and keep the connection that exported the snapshot open while the threads are exporting the data. For very large tables this process can take very long time (hours) which means that in case of an application error or closed connection we would need to restart the snapshot process. This is a critical process that might hold onto locks for longer than usual period of time (if it's not invoked from a `hotstandbyfeedback=off` replica). For